マスタ追加ダイアログUI設計書

メタデータ

title: マスタ追加ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-01-23
author: Development Team
tags: [data-admin, master, dialog, create, creator, vendor, instrument]

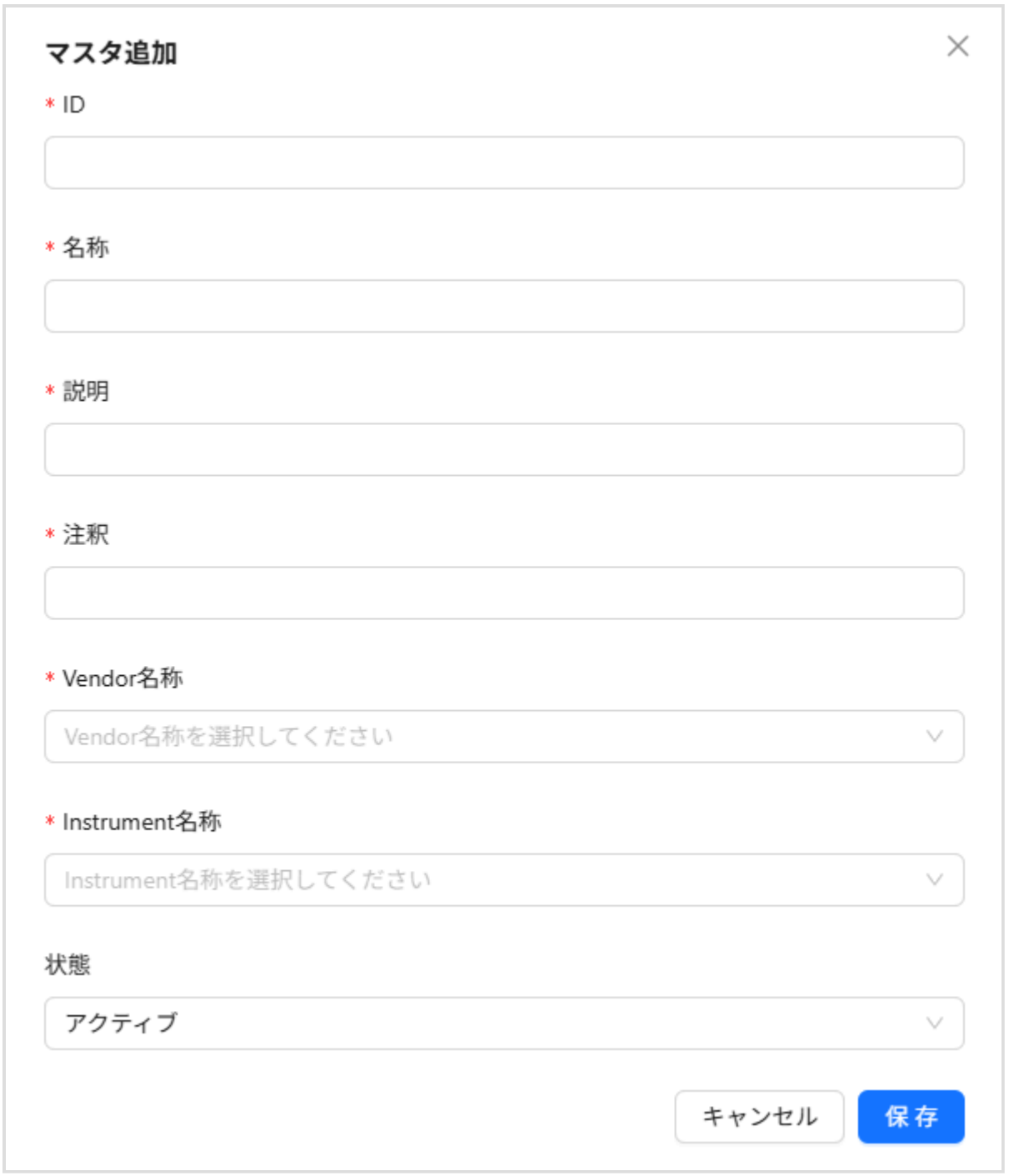
関連ドキュメント

- マスタ管理メイン画面
- マスタ編集ダイアログ
- マスタ管理API仕様
- CreatorテーブルDDL
- VendorテーブルDDL
- InstrumentテーブルDDL
- バックエンド実装計画
- UIポリシー

---

1. 画面レイアウト

1.1. 画面イメージ



2. 概要

2.1. 目的

データ管理者が新しいマスタデータ（Creator、Vendor、Instrument）を登録するためのダイアログを定義します。マスタ種別に応じた入力項目を表示し、マスタデータを追加します。

2.2. 利用権限

- **Data Admin（データ管理者）**: 使用可能

---

3. ダイアログ構成

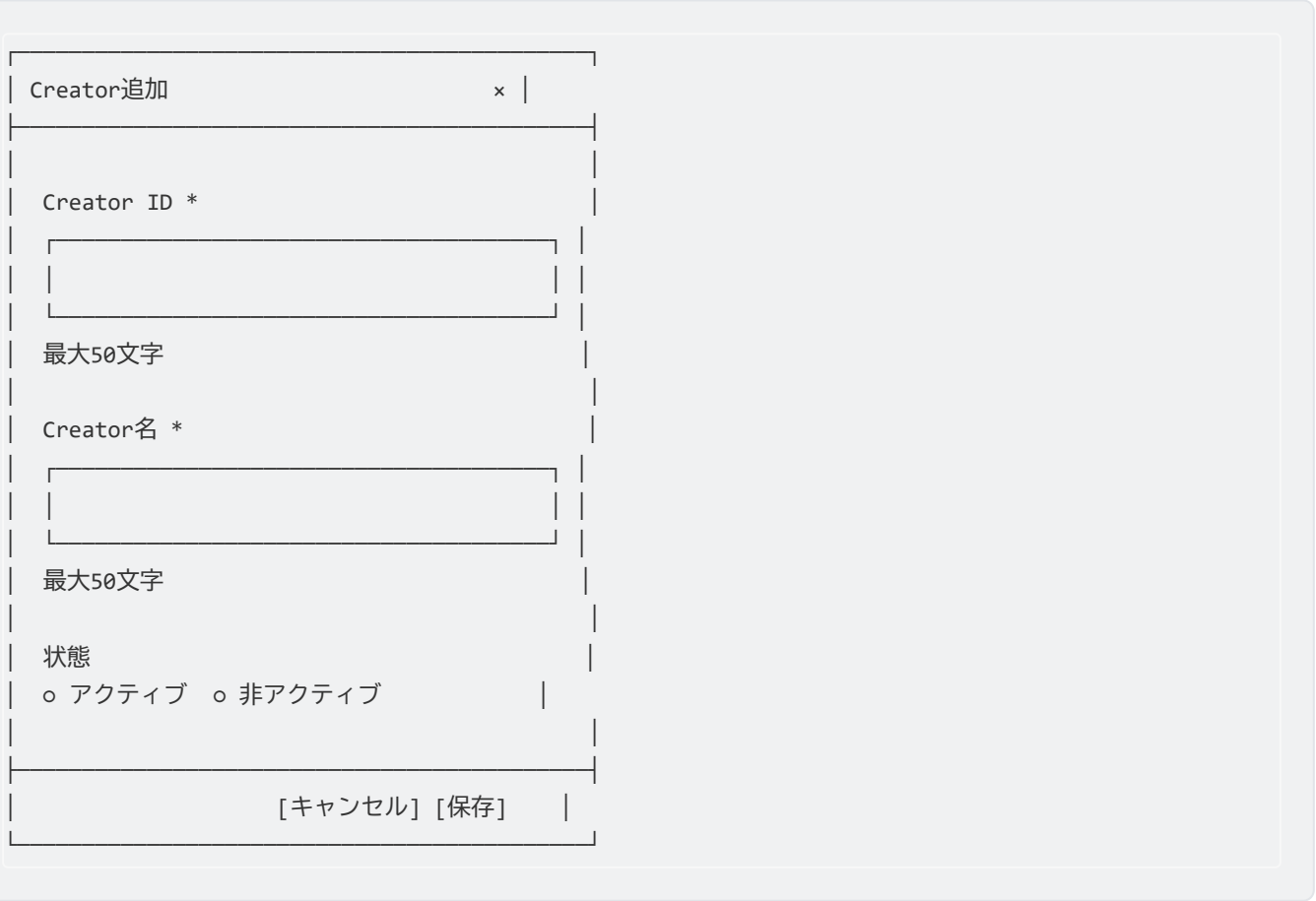
3.1. ダイアログ概要

表示トリガー: マスタ管理画面の「追加」ボタンクリック

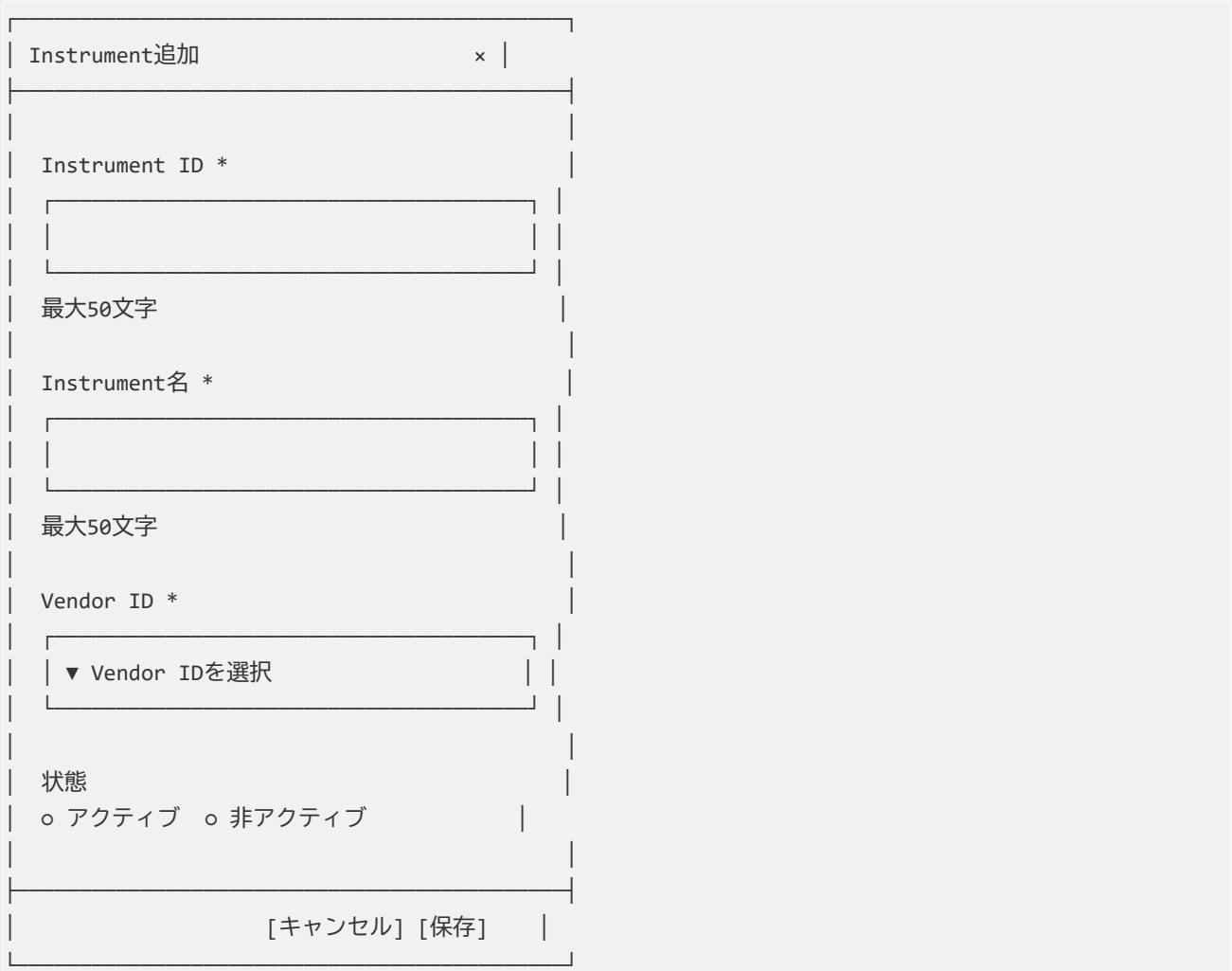
モーダルタイプ: Modal

サイズ: 中（600px）

3.2. レイアウト（Creator / Vendor の場合）



3.3. レイアウト（Instrument の場合）



---

4. 入力項目

4.1. マスタID（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	マスタ種別に応じて変動（「Creator ID」/「Vendor ID」/「Instrument ID」）
プレースホルダー	「IDを入力」

型	text
必須	○
最小文字数	1文字
最大文字数	50文字
許可文字	英数字、アンダースコア、ハイフン
重複チェック	有効（同一組織内）

バリデーション:

```
{
  required: true,
  message: 'IDを入力してください'
},
{
  min: 1,
  max: 50,
  message: 'IDは1〜50文字で入力してください'
},
{
  pattern: /^[a-zA-Z0-9_-]+$/,
  message: 'IDは英数字、アンダースコア、ハイフンのみ使用できます'
}
```

リアルタイム重複チェック:

- 入力後500msのデバウンス
- 既存のIDと重複する場合はエラー表示
- 背景色を黄色（`colorWarningBg`）に変更

4.2. マスタ名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	マスタ種別に応じて変動（「Creator名」/「Vendor名」/「Instrument名」）
プレースホルダー	「名前を入力」

型	text
必須	○
最小文字数	1文字
最大文字数	50文字

バリデーション:

```
{
  required: true,
  message: '名前を入力してください'
},
{
  min: 1,
  max: 50,
  message: '名前は1〜50文字で入力してください'
}
```

4.3. Vendor ID（Instrumentのみ必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「Vendor ID」
プレースホルダー	「Vendor IDを選択」
必須	○（Instrumentの場合のみ）
選択肢	アクティブなVendorマスタから取得
検索	有効（showSearch）
幅	100%

バリデーション:

```
{
  required: true,
```

```
  message: 'Vendor IDを選択してください'
}
```

選択肢の取得:

```
useEffect(() => {
  if (masterType === 'instrument') {
    fetchVendors();
  }
}, [masterType]);

const fetchVendors = async () => {
  try {
    const response = await apiClient.get('/api/v1/master/vendor', {
      params: { is_active: true },
      headers: { 'x-organization-id': organizationId }
    });

    const options = response.data.data.map((vendor: any) => ({
      label: `${vendor.vendorId} - ${vendor.vendorName}`,
      value: vendor.vendorId
    }));

    setVendorOptions(options);
  } catch (error) {
    message.error('Vendor一覧の取得に失敗しました');
  }
};
```

4.4. 状態（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「状態」
選択肢	アクティブ / 非アクティブ
デフォルト	アクティブ
必須	○

実装例:

```
<Form.Item
  label="状態"
  name="isActive"
  initialValue={true}
  rules={[{ required: true, message: '状態を選択してください' }]}
>
  <Radio.Group>
    <Radio value={true}>アクティブ</Radio>
    <Radio value={false}>非アクティブ</Radio>
  </Radio.Group>
</Form.Item>
```

---

5. アクションボタン

5.1. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`
配置	フッター左側

動作:

1. 入力内容を破棄
2. ダイアログを閉じる
3. 親画面に戻る

5.2. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
配置	フッター右側
ローディング	API呼び出し中は `true`
無効化	バリデーションエラー時

動作:

1. フォームバリデーション実行
2. ID重複チェック
3. API呼び出し: `POST /api/v1/master/{masterType}`
4. 成功時:
    - 成功メッセージ表示（Toast）
    - ダイアログを閉じる
    - 親画面の一覧を再取得
5. 失敗時: エラーメッセージ表示

---

6. APIリクエスト仕様

6.1. エンドポイント

`POST /api/v1/master/{masterType}`

マスタ種別パラメータ:

- `creator` - Creator マスタ
- `vendor` - Vendor マスタ
- `instrument` - Instrument マスタ

6.2. リクエストヘッダー

```
Content-Type: application/json
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

6.3. リクエストボディ

Creator:

```
{
  "creatorId": "CRT001",
  "creatorName": "Creator A",
  "isActive": true
}
```

Vendor:

```
{
  "vendorId": "VND001",
  "vendorName": "Vendor A",
  "isActive": true
}
```

Instrument:

```
{
  "instrumentId": "INS001",
  "instrumentName": "Instrument A",
  "vendorId": "VND001",
  "isActive": true
}
```

6.4. レスポンス（成功時）

Creator:


```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "creatorId": "CRT001",
    "creatorName": "Creator A",
    "isActive": true,
    "createAt": "2026-01-23T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```


Vendor:


```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "vendorId": "VND001",
    "vendorName": "Vendor A",
    "isActive": true,
    "createAt": "2026-01-23T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": { "..." }
}
```


Instrument:


```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "instrumentId": "INS001",
    "instrumentName": "Instrument A",
    "vendorId": "VND001",
    "isActive": true,
```

```
    "createAt": "2026-01-23T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": { "..." }
}
```

6.5. レスポンス（エラー時）

```
{
  "success": false,
  "error": {
    "code": "MASTER_ID_DUPLICATE",
    "message": "このIDは既に使用されています",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

---

7. エラーメッセージ

7.1. バリデーションエラー

エラー条件	メッセージ
IDが空	「IDを入力してください」
IDが長すぎる	「IDは1〜50文字で入力してください」
IDに不正な文字	「IDは英数字、アンダースコア、ハイフンのみ使用できます」
IDが重複	「このIDは既に使用されています」
名前が空	「名前を入力してください」
名前が長すぎる	「名前は1〜50文字で入力してください」
Vendor IDが未選択（Instrument）	「Vendor IDを選択してください」

7.2. APIエラー

エラーコード	メッセージ
`MASTER_ID_DUPLICATE`	「このIDは既に使用されています」
`VALIDATION_ERROR`	「入力内容に誤りがあります」
`VENDOR_NOT_FOUND`	「指定されたVendor IDが見つかりません」
`UNAUTHORIZED`	「この操作を実行する権限がありません」
`INTERNAL_SERVER_ERROR`	「サーバーエラーが発生しました。しばらく経ってから再試行してください」

---

8. 実装例（TypeScript + Next.js + Ant Design）

```tsx
// frontend/src/components/DataAdmin/MasterAddDialog.tsx
'use client';

import React, { useState, useEffect } from 'react';
import { Modal, Form, Input, Select, Radio, Button, message, theme } from 'antd';
import apiClient from '@/utils/apiClient';

const { useToken } = theme;

type MasterType = 'creator' | 'vendor' | 'instrument';

interface MasterAddDialogProps {
  open: boolean;
  onClose: () => void;
  onSuccess: () => void;
  masterType: MasterType;
  organizationId: string;
}

interface MasterFormValues {
  id: string;
  name: string;
  vendorId?: string;
  isActive: boolean;
}

const MASTER_LABELS = {
  creator: { id: 'Creator ID', name: 'Creator名', title: 'Creator追加' },
  vendor: { id: 'Vendor ID', name: 'Vendor名', title: 'Vendor追加' },
```

```
  instrument: { id: 'Instrument ID', name: 'Instrument名', title: 'Instrument追加' }
};

export default function MasterAddDialog({
  open,
  onClose,
  onSuccess,
  masterType,
  organizationId
}: MasterAddDialogProps) {
  const { token } = useToken();
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [isDuplicateChecking, setIsDuplicateChecking] = useState(false);
  const [isDuplicate, setIsDuplicate] = useState(false);
  const [vendorOptions, setVendorOptions] = useState<any[]>([]);

  const labels = MASTER_LABELS[masterType];

  // Vendor一覧取得（Instrumentの場合）
  useEffect(() => {
    if (open && masterType === 'instrument') {
      fetchVendors();
    }
  }, [open, masterType]);

  const fetchVendors = async () => {
    try {
      const response = await apiClient.get('/api/v1/master/vendor', {
        params: { is_active: true },
        headers: { 'x-organization-id': organizationId }
      });

      const options = response.data.data.map((vendor: any) => ({
        label: `${vendor.vendorId} - ${vendor.vendorName}`,
        value: vendor.vendorId
      }));

      setVendorOptions(options);
    } catch (error) {
      message.error('Vendor一覧の取得に失敗しました');
    }
  };

  // ID重複チェック（デバウンス付き）
  useEffect(() => {
    const id = form.getFieldValue('id');
    if (!id || id.length === 0) {
      setIsDuplicate(false);
      return;
```

```
    }

    const timer = setTimeout(async () => {
      setIsDuplicateChecking(true);
      try {
        const response = await apiClient.get(
          `/api/v1/master/${masterType}/${id}`,
          {
            headers: {
              'x-organization-id': organizationId
            }
          }
        );
        // 既に存在する場合
        if (response.data.success) {
          setIsDuplicate(true);
          form.setFields([
            {
              name: 'id',
              errors: ['このIDは既に使用されています']
            }
          ]);
        }
      } catch (error: any) {
        // 404の場合は重複なし
        if (error.response?.status === 404) {
          setIsDuplicate(false);
        }
      } finally {
        setIsDuplicateChecking(false);
      }
    }, 500);

    return () => clearTimeout(timer);
  }, [form.getFieldValue('id'), form, masterType, organizationId]);

  const handleSubmit = async (values: MasterFormValues) => {
    if (isDuplicate) {
      message.error('IDが重複しています');
      return;
    }

    setIsLoading(true);
    try {
      const requestBody: any = {
        isActive: values.isActive
      };

      // マスタ種別に応じてフィールド名を設定
      if (masterType === 'creator') {
```

```
        requestBody.creatorId = values.id;
        requestBody.creatorName = values.name;
      } else if (masterType === 'vendor') {
        requestBody.vendorId = values.id;
        requestBody.vendorName = values.name;
      } else if (masterType === 'instrument') {
        requestBody.instrumentId = values.id;
        requestBody.instrumentName = values.name;
        requestBody.vendorId = values.vendorId;
      }

      const response = await apiClient.post(
        `/api/v1/master/${masterType}`,
        requestBody,
        {
          headers: {
            'x-organization-id': organizationId
          }
        }
      );

      if (response.data.success) {
        message.success('マスタを追加しました');
        form.resetFields();
        onSuccess();
        onClose();
      }
    } catch (error: any) {
      const errorMessage =
        error.response?.data?.error?.message || 'マスタの追加に失敗しました';
      message.error(errorMessage);
    } finally {
      setIsLoading(false);
    }
  };

  const handleCancel = () => {
    form.resetFields();
    setIsDuplicate(false);
    onClose();
  };

  return (
    <Modal
      title={labels.title}
      open={open}
      onCancel={handleCancel}
      footer={null}
      width={600}
      destroyOnClose
```

```
    >
      <Form
        form={form}
        layout="vertical"
        onFinish={handleSubmit}
        initialValues={{ isActive: true }}
      >
        {/* ID */}
        <Form.Item
          label={labels.id}
          name="id"
          rules={[
            { required: true, message: 'IDを入力してください' },
            { min: 1, max: 50, message: 'IDは1〜50文字で入力してください' },
            {
              pattern: /^[a-zA-Z0-9_-]+$/,
              message: 'IDは英数字、アンダースコア、ハイフンのみ使用できます'
            }
          ]}
          extra="最大50文字"
          validateStatus={isDuplicate ? 'error' : undefined}
        >
          <Input
            placeholder="IDを入力"
            maxLength={50}
            style={{
              backgroundColor: isDuplicate ? token.colorWarningBg : undefined
            }}
            suffix={isDuplicateChecking ? <span>確認中...</span> : null}
          />
        </Form.Item>

        {/* 名前 */}
        <Form.Item
          label={labels.name}
          name="name"
          rules={[
            { required: true, message: '名前を入力してください' },
            { min: 1, max: 50, message: '名前は1〜50文字で入力してください' }
          ]}
          extra="最大50文字"
        >
          <Input placeholder="名前を入力" maxLength={50} />
        </Form.Item>

        {/* Vendor ID（Instrumentの場合のみ） */}
        {masterType === 'instrument' && (
          <Form.Item
            label="Vendor ID"
            name="vendorId"
```

```
            rules={[
              { required: true, message: 'Vendor IDを選択してください' }
            ]}
          >
            <Select
              placeholder="Vendor IDを選択"
              showSearch
              optionFilterProp="label"
              options={vendorOptions}
              style={{ width: '100%' }}
            />
          </Form.Item>
        )}

        {/* 状態 */}
        <Form.Item
          label="状態"
          name="isActive"
          rules={[{ required: true, message: '状態を選択してください' }]}
        >
          <Radio.Group>
            <Radio value={true}>アクティブ</Radio>
            <Radio value={false}>非アクティブ</Radio>
          </Radio.Group>
        </Form.Item>

        {/* フッター */}
        <Form.Item style={{ marginBottom: 0, marginTop: 24 }}>
          <div style={{ display: 'flex', justifyContent: 'flex-end', gap: 8 }}>
            <Button onClick={handleCancel}>キャンセル</Button>
            <Button
              type="primary"
              htmlType="submit"
              loading={isLoading}
              disabled={isDuplicate || isDuplicateChecking}
            >
              保存
            </Button>
          </div>
        </Form.Item>
      </Form>
    </Modal>
  );
}
```

---

9. テスト要件

9.1. 単体テスト

テストケース

- バリデーション機能のテスト
- 重複チェック機能のテスト
- フォーム送信処理のテスト
- Vendor一覧取得のテスト（Instrument）

9.2. 統合テスト

テストシナリオ

1. 正常なマスタ追加（Creator）
2. 正常なマスタ追加（Vendor）
3. 正常なマスタ追加（Instrument）
4. ID重複エラー
5. 必須項目未入力エラー
6. 最大文字数超過エラー
7. 不正な文字列入力エラー
8. Vendor ID未選択エラー（Instrument）

---

10. アクセシビリティ

10.1. キーボード操作

- Tab: 次の入力項目へフォーカス移動
- Shift + Tab: 前の入力項目へフォーカス移動
- Enter: フォーム送信
- Escape: ダイアログを閉じる

10.2. スクリーンリーダー対応

- 各入力項目に適切なラベルを設定
- エラーメッセージを `role="alert"` で通知
- 必須項目を `aria-required="true"` で明示

---

11. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-23	初版作成	Development Team